

```
@property
def remaining_seats(self) -> int:
    return self.seats - len(self.participants)
```

rend l'accès simple côté API.

Améliorations possibles

1. Éviter les places restantes négatives

Actuellement, si un bug permet d'inscrire trop de participants :

</> Python



▶ Exécuter

```
remaining_seats = -2
```

Vous pouvez sécuriser :

</> Python



▶ Exécuter

```
@property
def remaining_seats(self) -> int:
    return max(0, self.seats - len(self.participants))
```

2. Ajouter une validation du prix

Comme le prix est en centimes :

</> Python



▶ Exécuter

```
price: Mapped[int]
```

Il peut être négatif. Vous pouvez ajouter une contrainte SQL :

</> Python



▶ Exécuter

```
from sqlalchemy import CheckConstraint

class Course(db.Model):
    __tablename__ = "courses"

    __table_args__ = (
        CheckConstraint("price >= 0", name="price_non_negative"),
        CheckConstraint("seats >= 0", name="seats_non_negative"),
    )
```

3. Typage plus précis de la relation

Pour SQLAlchemy 2.x, vous pouvez écrire :

</> Python



▶ Exécuter

```
participants: Mapped[list["Participant"]] = relationship(
    "Participant",
    back_populates="course",
    cascade="all, delete-orphan"
)
```

Cela évite certains problèmes d'import circulaire.

4. Attention au calcul avec `len(self.participants)`

Cette ligne charge toute la liste des participants en mémoire :

</> Python



▶ Exécuter

```
len(self.participants)
```

Pour quelques dizaines de participants, aucun souci. Pour des milliers, mieux vaut compter côté SQL avec une requête.

5. Conversion du prix dans `to_dict`

Actuellement :

</> Python



▶ Exécuter

```
"price": self.price / 100,
```

renvoie un `float` :

</> JSON



```
{
  "price": 12.5
}
```

Si votre API attend toujours deux décimales :

</> Python



▶ Exécuter

```
"price": f"{self.price / 100:.2f}"
```

donnera :

```
</> JSON

{
  "price": "12.50"
}
```



ou utilisez un type décimal côté Python si nécessaire.

Une version légèrement renforcée pourrait être :

```
</> Python
```



▶ Exécuter

```
from database import db
from sqlalchemy.orm import Mapped, mapped_column, relationship
from sqlalchemy import String, Integer, CheckConstraint

class Course(db.Model):
    __tablename__ = "courses"

    __table_args__ = (
        CheckConstraint("price >= 0", name="price_non_negative"),
        CheckConstraint("seats >= 0", name="seats_non_negative"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)

    name: Mapped[str] = mapped_column(String(100), nullable=False)

    coach_name: Mapped[str] = mapped_column(String(100), nullable=False)

    seats: Mapped[int] = mapped_column(Integer, nullable=False)

    price: Mapped[int] = mapped_column(Integer, nullable=False)

    participants: Mapped[list["Participant"]] = relationship(
        "Participant",
        back_populates="course",
        cascade="all, delete-orphan"
    )

    @property
    def remaining_seats(self) -> int:
        return max(0, self.seats - len(self.participants))

    def to_dict(self) -> dict:
        return {
```

```
        "id": self.id,  
        "name": self.name,  
        "coach_name": self.coach_name,  
        "seats": self.seats,  
        "price": round(self.price / 100, 2),  
        "remaining_seats": self.remaining_seats  
    }
```

Le modèle est déjà bien structuré ; les changements ci-dessus concernent surtout la robustesse et la montée en charge.



met moi les commentaires pour expliquer le code

Voici ton code avec des commentaires explicatifs ligne par ligne :

</> Python



▶ Exécuter

```
# Import de l'objet db qui contient la configuration SQLAlchemy  
from database import db  
  
# Import des outils SQLAlchemy 2.x pour définir les types et colonnes des modèles  
from sqlalchemy.orm import Mapped, mapped_column, relationship  
  
# Import des types de colonnes SQL (texte, entier...)  
from sqlalchemy import String, Integer  
  
# Import du modèle Participant pour créer la relation entre un cours et ses participants  
from participants import Participant  
  
# Création du modèle Course qui représente la table "courses" dans la base de données  
class Course(db.Model):  
  
    # Nom de la table créée dans la base de données  
    __tablename__ = "courses"  
  
    # Identifiant unique du cours (clé primaire)  
    # Mapped[int] indique que cette colonne contient un entier  
    # mapped_column(primary_key=True) indique que c'est la clé primaire  
    id: Mapped[int] = mapped_column(primary_key=True)  
  
    # Nom du cours  
    # String(100) limite la taille du texte à 100 caractères
```

```
# nullable=False signifie que le champ est obligatoire
name: Mapped[str] = mapped_column(String(100), nullable=False)

# Nom du coach qui anime le cours
# Ce champ est également obligatoire
coach_name: Mapped[str] = mapped_column(String(100), nullable=False)

# Nombre maximum de places disponibles dans le cours
seats: Mapped[int] = mapped_column(Integer, nullable=False)

# Prix du cours stocké en centimes dans la base de données
# Exemple : 1500 correspond à 15,00 €
# Utiliser des centimes évite les erreurs d'arrondi avec les nombres décimaux
price: Mapped[int] = mapped_column(Integer, nullable=False)

# Création de la relation entre un cours et ses participants
# Un cours peut avoir plusieurs participants
# list["Participant"] indique qu'on récupère une liste de participants
participants: Mapped[list["Participant"]] = relationship(
    back_populates="course", # Permet de faire le lien avec l'attribut "course"
    cascade="all, delete-orphan" # Supprime automatiquement les participants liés
)

# Création d'une propriété calculée pour connaître le nombre de places restantes
# @property permet d'utiliser remaining_seats comme un attribut classique
# Exemple : course.remaining_seats
@property
def remaining_seats(self) -> int:

    # Calcul :
    # nombre total de places - nombre de participants inscrits
    return self.seats - len(self.participants)

# Fonction permettant de transformer un objet Course en dictionnaire Python
# Pratique pour envoyer les données sous forme JSON dans une API
def to_dict(self) -> dict:

    return {
        # Identifiant du cours
        "id": self.id,

        # Nom du cours
        "name": self.name,

        # Nom du coach
        "coach_name": self.coach_name,
```

```
# Nombre total de places disponibles
"seats": self.seats,

# Conversion du prix :
# la base contient des centimes (ex: 1500)
# l'API renvoie des euros (ex: 15.0)
"price": self.price / 100,

# Nombre de places encore disponibles
"remaining_seats": self.remaining_seats
}
```

J'ai gardé les commentaires dans un style adapté à un projet professionnel : ils expliquent **le rôle du code** plutôt que de répéter simplement ce que fait chaque ligne.



Cette conversation vous aide-t-elle jusqu'ici ?

